



**"RANKED 45
IN INDIA**

#University Category



**WORLD
UNIVERSITY
RANKINGS**

NO.1 PVT. UNIVERSITY IN
ACADEMIC REPUTATION IN INDIA



ACCREDITED GRADE 'A'
BY NAAC



PERFECT SCORE OF **150/150** AS A TESTAMENT
TO EXCEPTIONAL E-LEARNING METHODS

1



Applied Machine Learning

Unit 04 – Lecture 06: Feature Engineering and Reduction

Tofik Ali

School of Computer Science, UPES Dehradun

Autumn 2026

Scaling, encoding, PCA and feature selection

Topic focus: **the columns are a choice, not a given**

Repository: <https://github.com/tali7c/Applied-Machine-Learning>

Sources: Han, Kamber and Pei, *Data Mining: Concepts and Techniques* (3e), Ch. 3.; Müller and Guido, *Introduction to Machine Learning with Python*, Ch. 3–4.; James, Witten, Hastie, Tibshirani and Taylor, *An Introduction to Statistical Learning with Python*, Ch. 5–6, 12.; Scikit-learn documentation (preprocessing, model selection), accessed 2026-08-04.

Engineering
○○○○○

Scaling
○○○○○○○○○○○○

Encoding
○○○○○○○

Reduction
○○○○

PCA
○○○○○○○○○○○○○○

Selection
○○○○○○○○○

Summary
○○○

Quick Links

[Feature engineering](#)

[Scaling](#)

[Encoding](#)

[Data reduction](#)

[PCA](#)

[Feature selection](#)

[Summary](#)

Agenda

Feature engineering

Feature scaling

Feature encoding

Data reduction concepts

Dimensionality reduction: PCA

Feature selection

Summary

How this lecture works

Every idea appears twice

1. **By hand** — four plots of land, three customers, three colours, five points in the plane. Small enough to check on paper.
2. **In code** — the same arithmetic on real data, so you see that `StandardScaler` and `PCA` do exactly that and nothing more.

Commit before you look

Five slides ask you to predict a number before it is revealed. Write your guess down. Being wrong on paper is how the idea sticks.

The centrepiece is **a model that scores 0.85 on data with no signal in it whatever**.
Everything in the last section exists to make that failure obvious.

Where we are

Last lecture cleaned the table

Holes filled, stuck sensors found, long tails straightened. We ended on one rule: **every number used to clean the data must come from the training rows only.**

The table is now clean. It is not yet usable

The columns are in different units, some of them are words, and there may be far too many of them.

Today's four questions

What should the columns *be*? What *units* should they be in? What do I do with a column of *words*? And what if there are **too many columns**?

And the rule carries over unchanged

A scaler learns a mean. An encoder learns a list of categories. PCA learns a rotation. A selector learns *which columns*. All four are fitted on the training split.

What feature engineering is

Choosing the representation of the data that the model sees.

Four moves you will use

1. **Combine:** length and width become area.
2. **Decompose:** a timestamp becomes hour, weekday, month.
3. **Bin:** a continuous variable becomes ten intervals.
4. **Transform:** log, ratios, differences.

Where the leverage is

A linear model can only add its inputs up. If the thing that matters is a *product*, no amount of optimisation finds it — but one new column does.

Müller and Guido put it plainly: **how you represent your data matters more than which algorithm you pick.**

Four datasets, one seed

The data

Four bundled tables —
`load_breast_cancer`, `load_digits`,
`load_wine`, `load_diabetes` — plus small
problems drawn on the spot. Nothing is
downloaded.

The randomness

One seed: `np.random.default_rng(0)`,
`random_state=0` — including on
`mutual_info_classif`, which is
stochastic, and on the `StratifiedKFold`
shuffle.

The two exceptions are sweeps

The 40-split PCA experiment and the 20-repetition selection experiment change seed every repetition on purpose: the variation between repetitions is what they measure. Their seeds are derived from the same 0, so the sweep still reproduces exactly. Run the code as written and every number in this deck comes back.

By hand: four plots of land

Price is 5 per square metre, in rupees. Nobody recorded the area.

length (m)	10	20	30	40
width (m)	40	15	20	12
area = ℓw	400	300	600	480
price = $5\ell w$	2000	1500	3000	2400

corr(price, length)	+0.5494
corr(price, width)	-0.0948
corr(price, area)	+ 1.0000

Neither input is much use; their product is the answer

A linear model on (ℓ, w) reaches $R^2 = 0.6558$. Add the one column ℓw and it reaches 1.0000.
The information was always there. The representation was hiding it.

The same thing in code, on 400 rows

```
y = 3.0 * x1 * x2 + noise # the truth is a product
X_raw = np.c_[x1, x2]
X_eng = np.c_[x1, x2, x1 * x2] # one engineered column
```

```
linear model on x1, x2          cross-validated R2 0.9091
linear model on x1, x2, x1*x2   cross-validated R2 0.9981
a random forest on the raw pair gets there too, accuracy 0.9650
```

The same thing in code, on 400 rows

```
y = 3.0 * x1 * x2 + noise # the truth is a product
X_raw = np.c_[x1, x2]
X_eng = np.c_[x1, x2, x1 * x2] # one engineered column
```

```
linear model on x1, x2          cross-validated R2 0.9091
linear model on x1, x2, x1*x2   cross-validated R2 0.9981
a random forest on the raw pair gets there too, accuracy 0.9650
```

Read the first two lines

0.9091 → 0.9981. The remaining error is the noise floor. **One column of arithmetic.**

And read the third

A tree finds the interaction on its own, by splitting twice. **Feature engineering is worth most to the simple models.**

Binning: good for one model, bad for the other

One wiggly feature, 250 points, $y = \sin 4x + x + \varepsilon$. Replace x by ten KBinsDiscretizer intervals, one-hot encoded.

	raw x	10 bins
LinearRegression	0.8194	0.9151
DecisionTreeRegressor	0.9475	0.9151

binning bought the linear model +0.0957 and cost the tree -0.0323

Binning: good for one model, bad for the other

One wiggly feature, 250 points, $y = \sin 4x + x + \varepsilon$. Replace x by ten `KBinsDiscretizer` intervals, one-hot encoded.

	raw x	10 bins
LinearRegression	0.8194	0.9151
DecisionTreeRegressor	0.9475	0.9151

binning bought the linear model +0.0957 and cost the tree -0.0323

Both models now fit exactly the same function

After binning, a linear model on the ten indicator columns *is* a step function — the same family a tree fits. So the two scores are identical to four decimal places. **Binning gave the linear model the flexibility it lacked and took away the tree's ability to choose its own cut points.**

Binning: good for one model, bad for the other

One wiggly feature, 250 points, $y = \sin 4x + x + \varepsilon$. Replace x by ten KBinsDiscretizer intervals, one-hot encoded.

	raw x	10 bins
LinearRegression	0.8194	0.9151
DecisionTreeRegressor	0.9475	0.9151

binning bought the linear model +0.0957 and cost the tree -0.0323

Both models now fit exactly the same function

After binning, a linear model on the ten indicator columns *is* a step function — the same family a tree fits. So the two scores are identical to four decimal places. **Binning gave the linear model the flexibility it lacked and took away the tree's ability to choose its own cut points.**

This is Müller and Guido's point in one table: **a representation is good or bad only relative to a model.**

The obvious idea, and why it does not scale

PolynomialFeatures adds every product. How many columns is that?

d=2	degree 2 ->	5 features	d=2	degree 3 ->	9 features
d=10	degree 2 ->	65 features	d=10	degree 3 ->	285 features
d=30	degree 2 ->	495 features	d=30	degree 3 ->	5455 features
d=64	degree 2 ->	2144 features	d=64	degree 3 ->	47904 features

The obvious idea, and why it does not scale

PolynomialFeatures adds every product. How many columns is that?

d=2	degree 2 ->	5 features	d=2	degree 3 ->	9 features
d=10	degree 2 ->	65 features	d=10	degree 3 ->	285 features
d=30	degree 2 ->	495 features	d=30	degree 3 ->	5455 features
d=64	degree 2 ->	2144 features	d=64	degree 3 ->	47904 features

diabetes, ridge on 10 raw features	R2 0.4822
diabetes, ridge on 65 degree-2 features	R2 0.4179
difference	-0.0643

The obvious idea, and why it does not scale

PolynomialFeatures adds every product. How many columns is that?

d=2	degree 2 ->	5 features	d=2	degree 3 ->	9 features
d=10	degree 2 ->	65 features	d=10	degree 3 ->	285 features
d=30	degree 2 ->	495 features	d=30	degree 3 ->	5455 features
d=64	degree 2 ->	2144 features	d=64	degree 3 ->	47904 features

diabetes, ridge on 10 raw features	R2 0.4822
diabetes, ridge on 65 degree-2 features	R2 0.4179
difference	-0.0643

Engineering blind costs you six points of R^2

$\binom{d+k}{k} - 1$ grows like d^k . On 442 patients, 65 columns is already too many, and none of the products means anything clinically. **Engineer a feature because you have a reason, not because a function will generate one.**

Why the units of a column matter

Three mechanisms, one cause

1. **Distances.** k -NN, SVM, k -means add squared differences. A column measured in large numbers contributes large squares.
2. **Gradients.** Unequal scales make the loss surface a ravine, and descent zig-zags.
3. **Penalties.** Ridge and lasso shrink $\|w\|$, which means nothing if the w_j are in different units.

Who does not care

Decision trees, random forests, gradient boosting. A split asks “is $x_j \leq t$?”, and any *increasing* rescaling moves t without changing which rows go left.

Scaling is not transformation

L5's Box-Cox changed the **shape** of a distribution. Scaling changes only its **location and width**. The histogram keeps its outline.

By hand: three customers, age and income

	age (years)	annual income (in rupees)
A	25	400,000
B	60	405,000
C	26	800,000

Squared Euclidean distance, split into its two contributions:

$$d(A, B)^2 = \underbrace{35^2}_{1225} + \underbrace{5000^2}_{25,000,000}, \quad d(A, C)^2 = \underbrace{1^2}_1 + \underbrace{400,000^2}_{1.6 \times 10^{11}}.$$

Income supplies 99.995% of the first distance and 100.000% of the second

The age column is present in the file, present in the formula, and absent from the answer. **A 35-year age gap is worth less than a rounding error in the income column.**

The same three customers, standardised

$z = (x - \bar{x})/\sigma$, computed per column:

```
A age -0.7376 income -0.7204
B age +1.4138 income -0.6937
C age -0.6762 income +1.4141
scaled d(A,B)^2 = 4.6285 (age) + 0.0007 (income) = 4.6292 age is 99.98% of it
scaled d(A,C)^2 = 0.0038 (age) + 4.5562 (income) = 4.5600 age is 0.08% of it
nearest neighbour of A, raw : B
nearest neighbour of A, scaled : C
```

The same three customers, standardised

$z = (x - \bar{x})/\sigma$, computed per column:

```
A age -0.7376 income -0.7204
B age +1.4138 income -0.6937
C age -0.6762 income +1.4141
scaled d(A,B)^2 = 4.6285 (age) + 0.0007 (income) = 4.6292 age is 99.98% of it
scaled d(A,C)^2 = 0.0038 (age) + 4.5562 (income) = 4.5600 age is 0.08% of it
nearest neighbour of A, raw : B
nearest neighbour of A, scaled : C
```

The nearest neighbour of A changed

Raw, A's nearest neighbour is B — a 60-year-old, because their incomes happen to be close. Standardised, it is C — a 26-year-old. **Same data, same metric, different answer. Choosing not to scale is choosing an answer.**

Four scalers, four choices of centre and width

	Formula	Output	Broken by an outlier?
StandardScaler	$\frac{x - \bar{x}}{\sigma}$	mean 0, sd 1	yes: both $\bar{x}$ and σ move
MinMaxScaler	$\frac{x - x_{\min}}{x_{\max} - x_{\min}}$	exactly $[0, 1]$	completely : it <i>is</i> the extremes
RobustScaler	$\frac{x - \tilde{x}}{\text{IQR}}$	median 0, IQR 1	no: median and quartiles
MaxAbsScaler	$\frac{x}{\max x }$	$[-1, 1]$, keeps zeros	yes, but sparse-safe

None of them changes the shape

Every one is $x \mapsto (x - a)/b$. The histogram is translated and stretched. Skewness and kurtosis are **unchanged**. If the column is lopsided it is still lopsided — that was L5's job.

By hand: eight numbers through four scalers

$v = 2, 4, 4, 4, 5, 5, 7, 9$. Mean 5, $\sigma = 2$, min 2, max 9, median 4.5, $Q_1 = 4$, $Q_3 = 5.5$, IQR = 1.5.

```
StandardScaler [-1.5 -0.5 -0.5 -0.5  0.   0.   1.   2. ]
MinMaxScaler  [0.      0.2857 0.2857 0.2857 0.4286 0.4286 0.7143 1.    ]
RobustScaler  [-1.6667 -0.3333 -0.3333 -0.3333  0.3333 0.3333 1.6667 3.    ]
MaxAbsScaler  [0.2222 0.4444 0.4444 0.4444 0.5556 0.5556 0.7778 1.    ]
```

By hand: eight numbers through four scalers

$v = 2, 4, 4, 4, 5, 5, 7, 9$. Mean 5, $\sigma = 2$, min 2, max 9, median 4.5, $Q_1 = 4$, $Q_3 = 5.5$, IQR = 1.5.

```
StandardScaler [-1.5 -0.5 -0.5 -0.5  0.   0.   1.   2. ]
MinMaxScaler   [0.      0.2857 0.2857 0.2857 0.4286 0.4286 0.7143 1.    ]
RobustScaler   [-1.6667 -0.3333 -0.3333 -0.3333  0.3333 0.3333 1.6667 3.    ]
MaxAbsScaler   [0.2222 0.4444 0.4444 0.4444 0.5556 0.5556 0.7778 1.    ]
```

Check the first entry of each by hand: $(2 - 5)/2 = -1.5$; $(2 - 2)/(9 - 2) = 0$;
 $(2 - 4.5)/1.5 = -1.6667$; $2/9 = 0.2222$. **Four lines of arithmetic, no magic.** Note that
RobustScaler does *not* bound its output — 9 lands at 3.0.

Predict first #1

load_breast_cancer: 569 patients, 30 features whose standard deviations differ by a factor of 215,171. Five models, five-fold cross-validation, with and without StandardScaler.

Which of them move, and by how much?

Predict first #1

load_breast_cancer: 569 patients, 30 features whose standard deviations differ by a factor of 215,171. Five models, five-fold cross-validation, with and without StandardScaler.

Which of them move, and by how much?

model	raw	scaled	change
KNeighborsClassifier(5)	0.9315	0.9649	+0.0334
SVC(rbf)	0.9210	0.9771	+0.0561
LogisticRegression	0.9543	0.9789	+0.0246
DecisionTreeClassifier	0.9262	0.9262	+0.0000
RandomForestClassifier	0.9649	0.9649	+0.0000

Predict first #1

load_breast_cancer: 569 patients, 30 features whose standard deviations differ by a factor of 215,171. Five models, five-fold cross-validation, with and without StandardScaler.

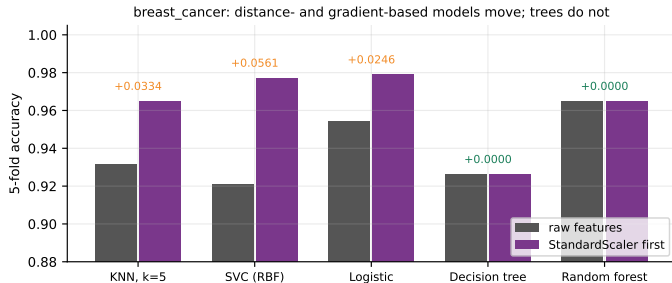
Which of them move, and by how much?

model	raw	scaled	change
KNeighborsClassifier(5)	0.9315	0.9649	+0.0334
SVC(rbf)	0.9210	0.9771	+0.0561
LogisticRegression	0.9543	0.9789	+0.0246
DecisionTreeClassifier	0.9262	0.9262	+0.0000
RandomForestClassifier	0.9649	0.9649	+0.0000

The two tree rows are +0.0000, not “about the same”

Five and a half points of accuracy for the RBF kernel, three for k -NN, two and a half for logistic regression — and **exactly zero, to every decimal place, for both tree models**. That contrast is the whole slide.

Drawn: who moves and who does not



Grey is raw, purple is standardised. The three left-hand pairs are the models that measure a distance or descend a gradient. The two right-hand pairs are the models that only ever ask *which side of a threshold*.

Why the tree bars are identical, exactly

```
t1 = DecisionTreeClassifier(random_state=0).fit(X_train, y_train)
sc = StandardScaler().fit(X_train)
t2 = DecisionTreeClassifier(random_state=0).fit(sc.transform(X_train),
        y_train)
```

```
tree on raw      : 18 leaves, depth 5, test accuracy 0.902098
tree on scaled  : 18 leaves, depth 5, test accuracy 0.902098
identical predictions on the test set: True
identical tree structure (same feature at every node): True
the thresholds differ, of course: raw 106.1000  scaled -0.0273 (same split)
```

Why the tree bars are identical, exactly

```
t1 = DecisionTreeClassifier(random_state=0).fit(X_train, y_train)
sc = StandardScaler().fit(X_train)
t2 = DecisionTreeClassifier(random_state=0).fit(sc.transform(X_train),
        y_train)
```

```
tree on raw      : 18 leaves, depth 5, test accuracy 0.902098
tree on scaled  : 18 leaves, depth 5, test accuracy 0.902098
identical predictions on the test set: True
identical tree structure (same feature at every node): True
the thresholds differ, of course: raw 106.1000  scaled -0.0273 (same split)
```

Not “similar” — **the same tree**. A split is a question about *order*, and $x \mapsto (x - a)/b$ with $b > 0$ preserves order. The threshold moves from 106.1 to -0.0273 and separates the same patients.

Predict first #2

The same eight numbers 2, 4, 4, 4, 5, 5, 7, 9. Somebody types one extra value: 500.

What happens to the eight good values under each scaler?

Predict first #2

The same eight numbers 2, 4, 4, 4, 5, 5, 7, 9. Somebody types one extra value: 500.

What happens to the eight good values under each scaler?

```
StandardScaler  the 8 good values now span 0.0450 (was 3.5000)
MinMaxScaler   the 8 good values now span 0.0141 (was 1.0000)
RobustScaler    the 8 good values now span 2.3333 (was 4.6667)
```

compression factor suffered by the 8 good values

```
StandardScaler  77.8 times narrower
MinMaxScaler    71.1 times narrower
RobustScaler    2.0 times narrower
```


Predict first #2

The same eight numbers 2, 4, 4, 4, 5, 5, 7, 9. Somebody types one extra value: 500.

What happens to the eight good values under each scaler?

```
StandardScaler  the 8 good values now span 0.0450 (was 3.5000)
MinMaxScaler    the 8 good values now span 0.0141 (was 1.0000)
RobustScaler     the 8 good values now span 2.3333 (was 4.6667)
```

compression factor suffered by the 8 good values

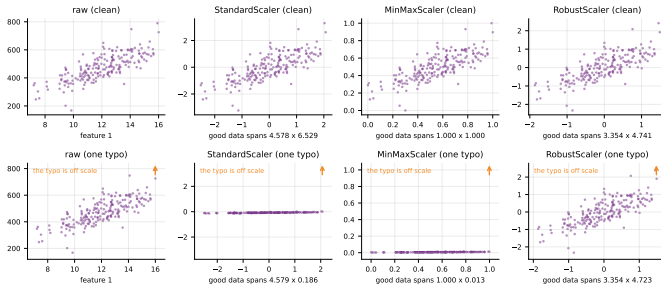
```
StandardScaler    77.8 times narrower
MinMaxScaler      71.1 times narrower
RobustScaler       2.0 times narrower
```

78×, **71×** and **2×**

The centre and width each scaler uses: mean 5 → 60, sd 2 → 155.6; max 9 → 500; **median** 4.5 → 5.0 **and IQR** 1.5 → 3.0. Only the third pair survived, because a median and a quartile do not care how far away a far-away point is.

Drawn: one typo, four scalers

Top: clean. Bottom: one area of 480 mistyped as 48000. Only RobustScaler leaves the good points spread out.



Two hundred plots of land; one area of 480 typed as 48000. The frames in the bottom row are the same frames as the top row. **Under MinMaxScaler the good data occupies 1.3% of the unit interval and every model downstream sees 200 identical rows.**

The gradient-based case, measured

SGDRegressor on load_diabetes in its original units versus standardised. Constant step size, up to 50,000 epochs.

eta0		raw units		standardised
0.01	6 epochs	R2 -1.7e+28	15 epochs	R2 +0.4951
0.001	6 epochs	R2 -4.8e+26	14 epochs	R2 +0.5142
0.0001	8 epochs	R2 -4.2e+25	184 epochs	R2 +0.5144
1e-05	24 epochs	R2 +0.2008	872 epochs	R2 +0.5139
1e-06	42 epochs	R2 +0.3478	5231 epochs	R2 +0.5113
ordinary least squares on the same data gives R2 +0.5177				

The gradient-based case, measured

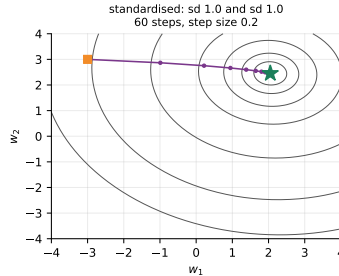
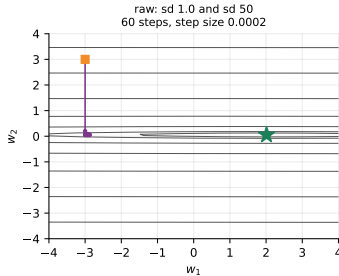
SGDRegressor on load_diabetes in its original units versus standardised. Constant step size, up to 50,000 epochs.

eta0		raw units		standardised
0.01	6 epochs	R2 -1.7e+28	15 epochs	R2 +0.4951
0.001	6 epochs	R2 -4.8e+26	14 epochs	R2 +0.5142
0.0001	8 epochs	R2 -4.2e+25	184 epochs	R2 +0.5144
1e-05	24 epochs	R2 +0.2008	872 epochs	R2 +0.5139
1e-06	42 epochs	R2 +0.3478	5231 epochs	R2 +0.5113
ordinary least squares on the same data gives R2 +0.5177				

There is no step size at which the raw version works

Large steps **diverge** ($R^2 = -10^{28}$); small steps stop early at 0.35. Standardised, every step size lands within 0.007 of least squares, and the best does it in **14 epochs**. Condition number of $X^T X$: 1.03×10^6 raw against 470 standardised.

Drawn: the ravine



Two features, one with fifty times the spread of the other. Orange square is the start, green star the least-squares answer. Left: the contours are a ravine (Hessian condition number 2296.5) and 60 steps end $323\times$ **worse than optimal**. Right: condition number 1.1, and 60 steps arrive.

A column of words is not a column of numbers

Two kinds of categorical variable

Ordinal — the categories have a real order. `small < medium < large`; a degree classification; a Likert scale.

Nominal — they do not. Colour, city, blood group, department, payment method.

Two encodings

Ordinal — one integer column, `0, 1, 2, ...`

One-hot — one 0/1 column per category, exactly one of them 1 in each row.

The decision

Match the encoding to the *variable*, not to your convenience. Ordinal-encoding a nominal variable asserts an order that is not there — and a linear model believes assertions.

By hand: three colours, ordinal-coded

OrdinalEncoder sorts alphabetically: blue→ 0, green→ 1, red→ 2. The true mean target for each colour is 20, 50, 10.

	blue = 0	green = 1	red = 2
true mean	20	50	10
best straight line	31.67	26.67	21.67

Least squares on three points: $\bar{x} = 1$, $\bar{y} = 26.667$, $S_{xy} = -10$, $S_{xx} = 2$, so slope = -5 and intercept = 31.667.

$$R^2 = 1 - \frac{816.7}{866.7} = \mathbf{0.0577}.$$

The line is forced to put green between the others

A line through the codes 0, 1, 2 *cannot* put the middle category highest. It captures 5.8% of the variation between three perfectly separated groups.

Predict first #3

900 rows, one nominal column, three colours with mean targets 20, 50 and 10 and a noise sd of 2. **The colour determines the target almost exactly.**

What cross-validated R^2 does a linear model get with ordinal coding? With one-hot?

Predict first #3

900 rows, one nominal column, three colours with mean targets 20, 50 and 10 and a noise sd of 2. **The colour determines the target almost exactly.**

What cross-validated R^2 does a linear model get with ordinal coding? With one-hot?

```
linear model, ordinal (1 column)    cross-validated R2 +0.0743
linear model, one-hot (3 columns)    cross-validated R2 +0.9868
a decision tree on the SAME ordinal column: accuracy 1.0000
```

Predict first #3

900 rows, one nominal column, three colours with mean targets 20, 50 and 10 and a noise sd of 2. **The colour determines the target almost exactly.**

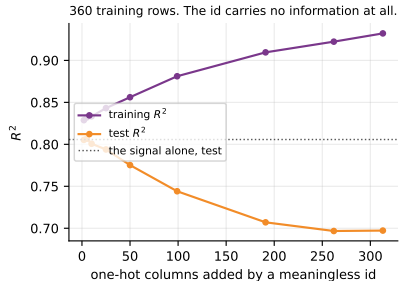
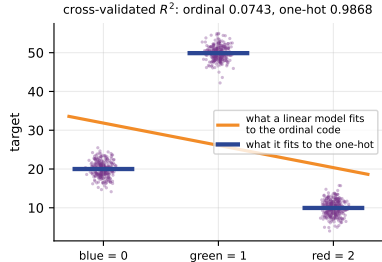
What cross-validated R^2 does a linear model get with ordinal coding? With one-hot?

```
linear model, ordinal (1 column)    cross-validated R2 +0.0743  
linear model, one-hot (3 columns)   cross-validated R2 +0.9868  
a decision tree on the SAME ordinal column: accuracy 1.0000
```

0.07 against 0.99, from a choice of encoder

The information is identical in both cases. Ordinal coding hid 92% of it from a linear model — and the hand calculation predicted 0.0577. **Read the third line too: a tree recovers it perfectly, because two splits on the code column separate three groups.**

Drawn: what each encoding lets the model say



Left: the orange line is everything a linear model can express about an ordinal code; the three blue bars are what one-hot lets it say. Right: the price of the opposite mistake — one-hot encoding an identifier that carries no information at all.

The dummy variable trap

Keep all three one-hot columns *and* an intercept:

```
with an intercept and all 3 dummies: shape (900, 4) rank 3
    the three dummy columns sum to [1.] in every row
    condition number 3.436e+15
with an intercept and 2 dummies:      shape (900, 3) rank 3
    condition number 3.810e+00
```

The dummy variable trap

Keep all three one-hot columns *and* an intercept:

```
with an intercept and all 3 dummies: shape (900, 4) rank 3
the three dummy columns sum to [1.] in every row
condition number 3.436e+15
```

```
with an intercept and 2 dummies:      shape (900, 3) rank 3
condition number 3.810e+00
```

```
coefficients, all 3 dummies : [ 19.9581  0.0382 29.926 -10.0061]
a DIFFERENT coefficient vector: [ 119.9581 -99.9618 -70.074 -110.0061]
same predictions? max difference 8.88e-15
```

The dummy variable trap

Keep all three one-hot columns *and* an intercept:

```
with an intercept and all 3 dummies: shape (900, 4) rank 3
the three dummy columns sum to [1.] in every row
condition number 3.436e+15
with an intercept and 2 dummies:      shape (900, 3) rank 3
condition number 3.810e+00
coefficients, all 3 dummies : [ 19.9581  0.0382 29.926 -10.0061]
a DIFFERENT coefficient vector: [ 119.9581 -99.9618 -70.074 -110.0061]
same predictions? max difference 8.88e-15
```

Add c to the intercept, subtract c from all three dummies

The predictions move by 9×10^{-15} , which is floating-point dust. There is no unique answer, so **the coefficients mean nothing and you must not interpret them**. Fix it with `drop='first'`: condition number $3.4 \times 10^{15} \rightarrow 3.81$. Trees and ridge do not care; least squares and every p -value do.

High cardinality: one-hot on an identifier

600 rows. One useful numeric feature. One customer id with 256 distinct values, generated at random and **carrying no information whatever**.

```
the id column has 256 distinct levels in 600 rows; one-hot makes 256 columns  
signal only          train R2 +0.7586    test R2 +0.8063  
signal + one-hot id  train R2 +0.8956    test R2 +0.6825  
levels present in train but not test: 99
```

High cardinality: one-hot on an identifier

600 rows. One useful numeric feature. One customer id with 256 distinct values, generated at random and **carrying no information whatever**.

```
the id column has 256 distinct levels in 600 rows; one-hot makes 256 columns  
signal only          train R2 +0.7586   test R2 +0.8063  
signal + one-hot id  train R2 +0.8956   test R2 +0.6825  
levels present in train but not test: 99
```

Training up, test down

+0.14 on the training set, −0.12 on the test set.
Each dummy fits a handful of rows exactly and predicts nothing.

What to do instead

Group rare levels into other; use a target or count encoding fitted *inside* the fold; or use a model that handles categories natively.

Two details that will bite you in the lab

```
enc = OneHotEncoder().fit([["small"], ["medium"], ["large"], ["small"]])  
enc.transform([["extra-large"]])
```

```
OneHotEncoder() default -> ValueError: Found unknown categories  
[np.str_('extra-large')] in column 0 during transform  
handle_unknown='ignore' -> all-zero row:  
[[0. 0. 1.]  
 [0. 0. 0.]]
```

Two details that will bite you in the lab

```
enc = OneHotEncoder().fit([["small"], ["medium"], ["large"], ["small"]])  
enc.transform([["extra-large"]])
```

```
OneHotEncoder() default -> ValueError: Found unknown categories  
[np.str_('extra-large')] in column 0 during transform  
handle_unknown='ignore' -> all-zero row:  
[[0. 0. 1.]  
 [0. 0. 0.]]
```

```
OrdinalEncoder().fit_transform(sizes) # ALPHABETICAL  
OrdinalEncoder(categories=[["small", "medium", "large"]]) # what you meant
```

```
alphabetical (the default): [2. 1. 0. 1. 2.]  
categories_ = ['large' 'medium' 'small']  
stated order           : [0. 1. 2. 1. 0.]
```

Two details that will bite you in the lab

```
enc = OneHotEncoder().fit([["small"], ["medium"], ["large"], ["small"]])  
enc.transform([["extra-large"]])
```

```
OneHotEncoder() default -> ValueError: Found unknown categories  
[np.str_('extra-large')] in column 0 during transform  
handle_unknown='ignore' -> all-zero row:  
[[0. 0. 1.]  
 [0. 0. 0.]]
```

```
OrdinalEncoder().fit_transform(sizes) # ALPHABETICAL  
OrdinalEncoder(categories=[["small", "medium", "large"]]) # what you meant
```

```
alphabetical (the default): [2. 1. 0. 1. 2.]  
categories_ = ['large' 'medium' 'small']  
stated order           : [0. 1. 2. 1. 0.]
```

The default put large before medium. If your variable really is ordinal, you must say so; the encoder cannot read English.

Han, Kamber and Pei's three kinds of reduction

	What it removes	Methods
Dimensionality	columns	attribute subset selection, PCA, wavelet transforms
Numerosity	rows, or the raw values	<i>parametric</i> : fit a model and keep its parameters; <i>non-parametric</i> : histograms, clustering, sampling
Compression	bits	lossless (run-length, entropy coding); lossy (PCA, wavelets again)

The test for all three

The reduced dataset must support the same analysis and give substantially the same answer, and the reduction itself must cost less than the saving. Today: dimensionality reduction (PCA) and attribute subset selection (feature selection), with one measurement on sampling.

Why fewer columns, though? The curse, measured

500 points spread uniformly in a cube of dimension d . For each point look at its nearest and its farthest neighbour.

d	mean dnear	mean dfar	dfar/dnear
2	0.0232	1.0332	67.5754
10	0.5495	1.8870	3.5278
50	2.1608	3.5227	1.6329
100	3.3785	4.7283	1.4004
500	8.4293	9.7936	1.1620
1000	12.2254	13.6046	1.1129

Why fewer columns, though? The curse, measured

500 points spread uniformly in a cube of dimension d . For each point look at its nearest and its farthest neighbour.

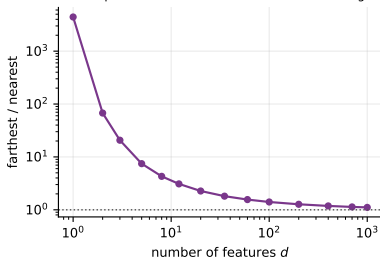
d	mean	d_{near}	mean	d_{far}	$d_{\text{far}}/d_{\text{near}}$
2		0.0232		1.0332	67.5754
10		0.5495		1.8870	3.5278
50		2.1608		3.5227	1.6329
100		3.3785		4.7283	1.4004
500		8.4293		9.7936	1.1620
1000		12.2254		13.6046	1.1129

At $d = 1000$ the farthest point is $1.11\times$ as far as the nearest

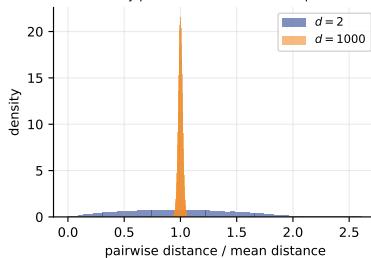
Every pair of points is at very nearly the same distance, so “nearest neighbour” stops carrying information. **k -NN, k -means and every kernel with a distance in it degrade as d grows — and they degrade quietly.**

Drawn: distances concentrate

500 uniform points: how much farther the farthest neighbour is



every pair is the same distance apart



Left: the ratio falls towards 1 on a log-log scale. Right: the whole distribution of pairwise distances at $d = 2$ and $d = 1000$, each divided by its own mean. **At $d = 1000$ every distance in the dataset is within a few per cent of every other.**

Numerosity reduction: keep fewer rows

load_digits, 1797 rows. Take a random sample and refit.

digits, all 1797 rows	accuracy 0.9694		
digits, a 50% sample (898 rows)	accuracy 0.9607	loss -0.0087	
digits, a 25% sample (449 rows)	accuracy 0.9474	loss -0.0220	
digits, a 10% sample (179 rows)	accuracy 0.9134	loss -0.0560	
digits, a 5% sample (89 rows)	accuracy 0.8506	loss -0.1188	

Numerosity reduction: keep fewer rows

load_digits, 1797 rows. Take a random sample and refit.

digits, all 1797 rows	accuracy 0.9694		
digits, a 50% sample (898 rows)	accuracy 0.9607	loss -0.0087	
digits, a 25% sample (449 rows)	accuracy 0.9474	loss -0.0220	
digits, a 10% sample (179 rows)	accuracy 0.9134	loss -0.0560	
digits, a 5% sample (89 rows)	accuracy 0.8506	loss -0.1188	

Half the data costs **under one point**. A twentieth costs twelve. **Sampling is the cheapest reduction there is and it is often free at the top of the curve** — which is why you should prototype on a sample and fit on everything.

Numerosity reduction: keep fewer rows

load_digits, 1797 rows. Take a random sample and refit.

digits, all 1797 rows	accuracy 0.9694		
digits, a 50% sample (898 rows)	accuracy 0.9607	loss -0.0087	
digits, a 25% sample (449 rows)	accuracy 0.9474	loss -0.0220	
digits, a 10% sample (179 rows)	accuracy 0.9134	loss -0.0560	
digits, a 5% sample (89 rows)	accuracy 0.8506	loss -0.1188	

Half the data costs **under one point**. A twentieth costs twelve. **Sampling is the cheapest reduction there is and it is often free at the top of the curve** — which is why you should prototype on a sample and fit on everything.

Sampling is also how you make a 200-fold experiment finish before the lab ends.

What PCA is asking

Of all the directions in feature space, which one do the data **vary along the most**?
Then, of the directions perpendicular to that one, which is next?

The recipe

1. Centre every column (subtract its mean).
2. Form the covariance matrix
$$S = \frac{1}{n-1} X_c^T X_c.$$
3. Take its eigenvectors and eigenvalues.
4. Sort by eigenvalue, keep the top k .

Two facts to remember

λ_j is the variance along component j , so
 $\lambda_j / \sum_i \lambda_i$ is the fraction of variance explained.
And $\sum_i \lambda_i = \text{tr}(S)$, the total variance — the
rotation moves variance around but never
creates or destroys it.

Equivalently: PCA finds the k -dimensional subspace with the **smallest squared reconstruction error**. Same answer, different question.

By hand: five points in the plane

(1, 2), (2, 3), (3, 5), (4, 4), (5, 6). Mean (3, 4). Centred:
(-2, -2), (-1, -1), (0, 1), (1, 0), (2, 2).

$$S = \frac{1}{4} \begin{pmatrix} 10 & 9 \\ 9 & 10 \end{pmatrix} = \begin{pmatrix} 2.5 & 2.25 \\ 2.25 & 2.5 \end{pmatrix}.$$

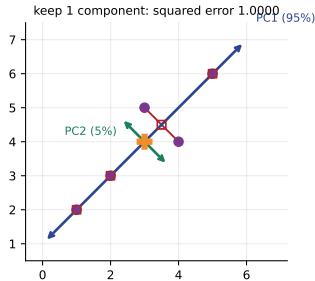
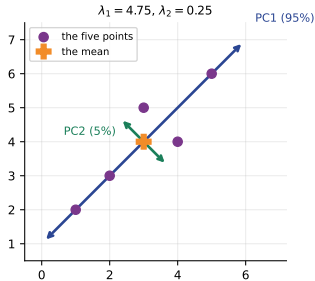
For a symmetric matrix with equal diagonal entries the eigenvectors are $\frac{1}{\sqrt{2}}(1, 1)$ and $\frac{1}{\sqrt{2}}(1, -1)$, with eigenvalues 2.5 ± 2.25 :

$$\lambda_1 = 4.75, \quad \lambda_2 = 0.25, \quad \frac{\lambda_1}{\lambda_1 + \lambda_2} = \frac{4.75}{5.00} = \mathbf{0.95}.$$

Check the trace

$2.5 + 2.5 = 5.0 = 4.75 + 0.25$. The two original columns had variance 2.5 each; the two components have 4.75 and 0.25. **Nothing was created; it was rearranged.**

Drawn: the same five points



Arrows are the eigenvectors, drawn at length $\sqrt{\lambda_j}$. Right: each point projected onto PC1 alone. Only two points move, by $\pm(0.5, -0.5)$, so the total squared error is $0.5 + 0.5 = 1.0$ — which is exactly $(n - 1)\lambda_2 = 4 \times 0.25$.

The same thing in code

```
S = np.cov(P - P.mean(axis=0), rowvar=False)
w, V = np.linalg.eigh(S) # by hand
p = PCA(n_components=2).fit(P) # by library
```

```
sklearn explained_variance_      [4.75 0.25]
sklearn explained_variance_ratio_ [0.95 0.05]
sklearn components_ (rows)
[[ 0.707107  0.707107]
 [ 0.707107 -0.707107]]
reconstruction from 1 component [[1.0, 2.0], [2.0, 3.0], [3.5, 4.5], [3.5, 4.5], [5.0, 6.0]]
total squared reconstruction error 1.000000
check: (n-1) * lambda2 = 4 * 0.2500 = 1.000000
```

The same thing in code

```
S = np.cov(P - P.mean(axis=0), rowvar=False)
w, V = np.linalg.eigh(S) # by hand
p = PCA(n_components=2).fit(P) # by library
```

```
sklearn explained_variance_      [4.75 0.25]
sklearn explained_variance_ratio_ [0.95 0.05]
sklearn components_ (rows)
  [[ 0.707107  0.707107]
   [ 0.707107 -0.707107]]
reconstruction from 1 component [[1.0, 2.0], [2.0, 3.0], [3.5, 4.5], [3.5, 4.5], [5.0, 6.0]]
total squared reconstruction error 1.000000
check: (n-1) * lambda2 = 4 * 0.2500 = 1.000000
```

Identical to the hand calculation. **Note that the two points (3,5) and (4,4) both reconstruct to (3.5,4.5)** — dropping a component merges points that differed only along it. That is what “lossy” means.

Predict first #4

Run `PCA()` on `load.breast.cancer` straight out of the box, without scaling. The columns include `mean radius` (around 14) and `worst area` (around 880).

How much of the variance does the first component explain, and what is it?

Predict first #4

Run `PCA()` on `load.breast_cancer` straight out of the box, without scaling. The columns include `mean radius` (around 14) and `worst area` (around 880).

How much of the variance does the first component explain, and what is it?

```
PCA on breast_cancer WITHOUT scaling
PC1 explains 0.9820 of the variance
its largest loading is 0.8521 on 'worst area'
that feature's standard deviation is 568.9; the smallest in the table is 0.00264
PCA on breast_cancer WITH StandardScaler
PC1 explains 0.4427 of the variance
its largest loading is 0.2609 on 'mean concave points'
```

Predict first #4

Run `PCA()` on `load.breast_cancer` straight out of the box, without scaling. The columns include `mean radius` (around 14) and `worst area` (around 880).

How much of the variance does the first component explain, and what is it?

```
PCA on breast_cancer WITHOUT scaling
PC1 explains 0.9820 of the variance
its largest loading is 0.8521 on 'worst area'
that feature's standard deviation is 568.9; the smallest in the table is 0.00264
PCA on breast_cancer WITH StandardScaler
PC1 explains 0.4427 of the variance
its largest loading is 0.2609 on 'mean concave points'
```

98.2% — and it is just the area column, renamed

PCA maximises *variance*, and variance has units. The widest column wins by construction. **Standardise before PCA unless every column is already in the same unit**, or you have run an expensive eigendecomposition to discover which column you measured in the biggest numbers.

How many components? Read the curve

breast_cancer, standardised, 30 features

1 components explain 44.27%	80% needs 5 components (16.7% of 30)
-----------------------------	--------------------------------------

2 components explain 63.24%	90% needs 7 components (23.3% of 30)
-----------------------------	--------------------------------------

5 components explain 84.73%	95% needs 10 components (33.3% of 30)
-----------------------------	---------------------------------------

10 components explain 95.16%	99% needs 17 components (56.7% of 30)
------------------------------	---------------------------------------

digits, standardised, 64 features

80% needs 21	90% needs 31	95% needs 40	99% needs 54 of 64
--------------	--------------	--------------	--------------------

How many components? Read the curve

breast_cancer, standardised, 30 features

1 components explain 44.27% 80% needs 5 components (16.7% of 30)

2 components explain 63.24% 90% needs 7 components (23.3% of 30)

5 components explain 84.73% 95% needs 10 components (33.3% of 30)

10 components explain 95.16% 99% needs 17 components (56.7% of 30)

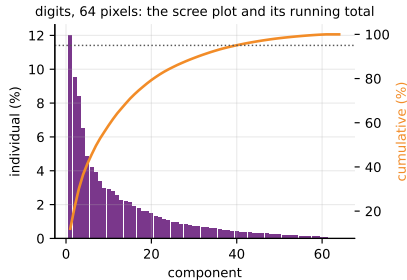
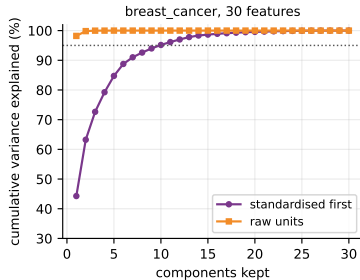
digits, standardised, 64 features

80% needs 21 90% needs 31 95% needs 40 99% needs 54 of 64

Compare the two datasets before you generalise

breast_cancer compresses beautifully: ten of thirty columns carry 95%, because the thirty features are ten measurements repeated three ways. Digits does not: **forty of sixty-four**. There is no universal “keep 95%” rule — 95% of what depends entirely on the data.

Drawn: variance explained, and what scaling does to it



Left: the orange curve starts at 98.2% because one unscaled column dominates; the purple curve is the honest one. Right: the digits scree plot with its running total. **An elbow is a suggestion, not a criterion.**

What you actually lose: reconstruct the digits

Each digit is $8 \times 8 = 64$ pixels. Project to k components and come back.

k= 1	variance kept	14.89%	relative reconstruction error	85.11%
k= 2	variance kept	28.51%	relative reconstruction error	71.49%
k= 5	variance kept	54.50%	relative reconstruction error	45.50%
k=10	variance kept	73.82%	relative reconstruction error	26.18%
k=20	variance kept	89.43%	relative reconstruction error	10.57%
k=40	variance kept	98.82%	relative reconstruction error	1.18%
k=64	variance kept	100.00%	relative reconstruction error	0.00%

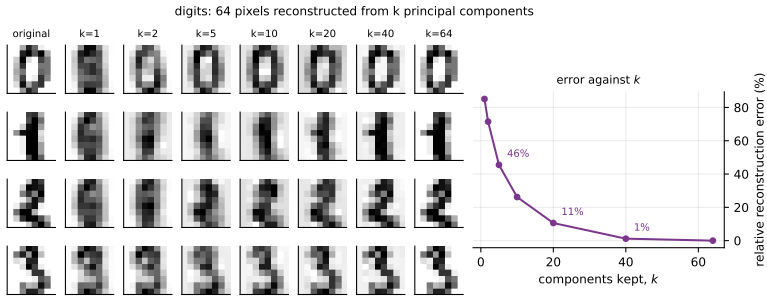
What you actually lose: reconstruct the digits

Each digit is $8 \times 8 = 64$ pixels. Project to k components and come back.

k= 1	variance kept	14.89%	relative reconstruction error	85.11%
k= 2	variance kept	28.51%	relative reconstruction error	71.49%
k= 5	variance kept	54.50%	relative reconstruction error	45.50%
k=10	variance kept	73.82%	relative reconstruction error	26.18%
k=20	variance kept	89.43%	relative reconstruction error	10.57%
k=40	variance kept	98.82%	relative reconstruction error	1.18%
k=64	variance kept	100.00%	relative reconstruction error	0.00%

Variance kept and error removed add to exactly 100%. That is not a coincidence: the reconstruction error of the top k components is $\sum_{j>k} \lambda_j$, and dividing by $\sum_j \lambda_j$ gives $1 - (\text{variance kept})$.

Drawn: a digit, rebuilt from k numbers



At $k = 5$ you can already tell a 0 from a 1; at $k = 20$ the digits are legible and 44 of the 64 numbers per image have been thrown away. **The error curve is the same $\sum_{j>k} \lambda_j$ read from the other side.**

PCA is not feature selection

The first component of the standardised breast_cancer data:

```
largest |loading| 0.2609    smallest |loading| 0.0145
number of features with |loading| > 0.10 : 26 of 30
+0.2609 mean concave points +0.2584 mean concavity
+0.2509 worst concave points +0.2393 mean compactness
to compute PC1 for a new patient you need ALL 30 measurements.
```

PCA is not feature selection

The first component of the standardised breast_cancer data:

```
largest |loading| 0.2609    smallest |loading| 0.0145
number of features with |loading| > 0.10 : 26 of 30
    +0.2609 mean concave points    +0.2584 mean concavity
    +0.2509 worst concave points    +0.2393 mean compactness
to compute PC1 for a new patient you need ALL 30 measurements.
```

Ten components does not mean ten measurements

Every component is a weighted sum of all thirty columns. You have reduced the *width of the matrix* and not the *cost of collecting a row*. **If your reason for reducing was “this test is expensive”, PCA has not helped you at all** — feature selection has, and that is the next section.

PCA is not feature selection

The first component of the standardised breast_cancer data:

```
largest |loading| 0.2609    smallest |loading| 0.0145
number of features with |loading| > 0.10 : 26 of 30
    +0.2609 mean concave points    +0.2584 mean concavity
    +0.2509 worst concave points    +0.2393 mean compactness
to compute PC1 for a new patient you need ALL 30 measurements.
```

Ten components does not mean ten measurements

Every component is a weighted sum of all thirty columns. You have reduced the *width of the matrix* and not the *cost of collecting a row*. **If your reason for reducing was “this test is expensive”, PCA has not helped you at all** — feature selection has, and that is the next section.

And $0.2609 \times \text{concave points} + 0.2584 \times \text{concavity} + \dots$ is not a quantity a clinician can name.

And PCA has never heard of your labels

Two features: one with sd 10 and no signal, one with sd 1 that *is* the label.

```
PC1 explains 0.9909 of the variance; its loadings are [1.    0.0052]
logistic regression on both features          accuracy 0.9850
logistic regression on PCA's 1 component      accuracy 0.5200
logistic regression on the 1 feature a filter would pick 0.9933
after standardising first, PCA's component    accuracy 0.7617
```

And PCA has never heard of your labels

Two features: one with sd 10 and no signal, one with sd 1 that *is* the label.

```
PC1 explains 0.9909 of the variance; its loadings are [1.      0.0052]
logistic regression on both features          accuracy 0.9850
logistic regression on PCA's 1 component      accuracy 0.5200
logistic regression on the 1 feature a filter would pick 0.9933
after standardising first, PCA's component    accuracy 0.7617
```

PCA threw away the answer and kept the noise

0.9850 $\rightarrow$ 0.5200, which is a coin flip. PCA is **unsupervised**: it maximises variance, and variance is not relevance. A supervised filter picking one column got 0.9933. **Standardising first helps (0.7617) but does not fix it**, because the problem is not the units — it is that y was never consulted.

PCA is fitted on the training split. Only.

Three ways to run the same experiment, 40 random splits of breast_cancer, 10 components:

```
correct                accuracy 0.9768 +/- 0.0128 worst split 0.9357
fit on everything       accuracy 0.9773 +/- 0.0122 worst split 0.9415
refit on the test set   accuracy 0.9447 +/- 0.0226 worst split 0.8947
fit-on-everything minus correct: +0.0006 t +0.94 wins 8/40
components whose sign flipped between the two fits: 3.0 of 10, up to 6
```

PCA is fitted on the training split. Only.

Three ways to run the same experiment, 40 random splits of breast_cancer, 10 components:

```
correct                accuracy 0.9768 +/- 0.0128  worst split 0.9357
fit on everything       accuracy 0.9773 +/- 0.0122  worst split 0.9415
refit on the test set  accuracy 0.9447 +/- 0.0226  worst split 0.8947
fit-on-everything minus correct: +0.0006  t +0.94  wins 8/40
components whose sign flipped between the two fits: 3.0 of 10, up to 6
```

Be honest about which of the two mistakes is which

Fitting PCA on all the rows before splitting leaked **nothing measurable** here ($t = 0.94$) — PCA never sees y , so there is little to leak. Calling `fit_transform` *again* on the test set is a different and much worse error: -0.0320 on average and -0.0877 at worst, because the test components are a different basis — three of the ten point the opposite way.

PCA is fitted on the training split. Only.

Three ways to run the same experiment, 40 random splits of breast_cancer, 10 components:

```
correct                accuracy 0.9768 +/- 0.0128   worst split 0.9357
fit on everything       accuracy 0.9773 +/- 0.0122   worst split 0.9415
refit on the test set   accuracy 0.9447 +/- 0.0226   worst split 0.8947
fit-on-everything minus correct: +0.0006  t +0.94  wins 8/40
components whose sign flipped between the two fits: 3.0 of 10, up to 6
```

Be honest about which of the two mistakes is which

Fitting PCA on all the rows before splitting leaked **nothing measurable** here ($t = 0.94$) — PCA never sees y , so there is little to leak. Calling `fit_transform` *again* on the test set is a different and much worse error: -0.0320 on average and -0.0877 at worst, because the test components are a different basis — three of the ten point the opposite way.

The version that leaks a great deal is coming in two slides.

Three families, one goal

	How it decides	Cost / risk
Filter	a statistic per feature, computed once, model ignored — variance, ANOVA F , mutual information, correlation	cheapest; blind to interactions and to redundancy
Wrapper	fit the model repeatedly and keep what helps — forward selection, backward elimination, RFE	most expensive; can overfit the selection
Embedded	the model selects while it fits — L1 (lasso), tree importances	nearly free; tied to that model

Why bother at all

Fewer columns means a cheaper model, a cheaper *row* to collect, less overfitting — and, unlike PCA, an answer you can read out loud. **Han, Kamber and Pei call this attribute subset selection.**

Filter, part 1: variance

`load_digits`: the corner pixels of an 8×8 scan are always blank.

`digits`: 3 of the 64 pixels have zero variance (always the same value)

48 of the 64 survive a variance threshold of 1.0

logistic regression on all 64 pixels

accuracy 0.9694

threshold 0.0 keeps 61 pixels

accuracy 0.9694

threshold 1.0 keeps 48 pixels

accuracy 0.9694

threshold 10.0 keeps 43 pixels

accuracy 0.9683

Filter, part 1: variance

`load_digits`: the corner pixels of an 8×8 scan are always blank.

`digits`: 3 of the 64 pixels have zero variance (always the same value)

48 of the 64 survive a variance threshold of 1.0

logistic regression on all 64 pixels	accuracy 0.9694
threshold 0.0 keeps 61 pixels	accuracy 0.9694
threshold 1.0 keeps 48 pixels	accuracy 0.9694
threshold 10.0 keeps 43 pixels	accuracy 0.9683

Sixteen of the sixty-four columns removed for nothing at all, and twenty-one for 0.0011. A variance filter almost never improves a score. What it buys is **columns** — and a constant column is not merely useless, it breaks `StandardScaler` (division by zero) and any correlation you compute.

Filter, part 2: where the two filters disagree

Ten features. *a* enters the label linearly; *b* enters only through $|b|$; the other eight are noise.

feature	ANOVA F	rank	mutual info	rank
<i>a</i> (linear)	317.110	1	0.1342	2
<i>b</i> ($ b $ only)	0.213	8	0.2263	1
best noise	8.326	-	0.0123	-
SelectKBest(f_classif, k=3) picks ['a', 'noise 1', 'noise 2']				
SelectKBest(mutual_info_classif, k=3) picks ['a', 'b', 'noise 8']				

Filter, part 2: where the two filters disagree

Ten features. a enters the label linearly; b enters only through $|b|$; the other eight are noise.

feature	ANOVA F	rank	mutual info	rank
a (linear)	317.110	1	0.1342	2
b ($ b $ only)	0.213	8	0.2263	1
best noise	8.326	–	0.0123	–
SelectKBest(f_classif, k=3) picks ['a', 'noise 1', 'noise 2']				
SelectKBest(mutual_info_classif, k=3) picks ['a', 'b', 'noise 8']				

The F test ranks the real feature eighth of ten

F compares *class means*. The two classes have the same mean b , so it sees nothing — and picks two noise columns ahead of it. Mutual information makes no linearity assumption and ranks b first. **A filter measures the kind of dependence it was built to measure, and nothing else.**

Wrapper: recursive feature elimination

Fit the model. Drop the feature with the smallest coefficient. Refit. Repeat. Thirty features with step=1 means **thirty model fits**.

```
elimination order, first six      the last one standing: worst area
dropped 30 of 30: worst compactness    dropped 27: mean smoothness
dropped 29 of 30: mean symmetry        dropped 26: texture error
dropped 28 of 30: concavity error      dropped 25: concave points error
RFE k= 1  accuracy 0.9157             RFE k=10  accuracy 0.9684
RFE k= 2  accuracy 0.9526             RFE k=20  accuracy 0.9789
RFE k= 5  accuracy 0.9684             RFE k=30  accuracy 0.9789
RFECV picks its own k: 26 features, accuracy 0.9824
```

Wrapper: recursive feature elimination

Fit the model. Drop the feature with the smallest coefficient. Refit. Repeat. Thirty features with `step=1` means **thirty model fits**.

```
elimination order, first six      the last one standing: worst area
dropped 30 of 30: worst compactness    dropped 27: mean smoothness
dropped 29 of 30: mean symmetry        dropped 26: texture error
dropped 28 of 30: concavity error      dropped 25: concave points error
RFE k= 1  accuracy 0.9157             RFE k=10  accuracy 0.9684
RFE k= 2  accuracy 0.9526             RFE k=20  accuracy 0.9789
RFE k= 5  accuracy 0.9684             RFE k=30  accuracy 0.9789
RFECV picks its own k: 26 features, accuracy 0.9824
```

A wrapper asks the *model* what it needs, so it can see redundancy a filter cannot — but it pays a model fit per step, and its answer is **that model's** answer. **Left to choose k for itself**, RFECV keeps 26 of the 30.

Embedded: L1 chooses by shrinking to zero

Lasso on load_diabetes, standardised, over a path of α :

```
alpha 0.01 10 nonzero CV R2 +0.4823
alpha 0.50 8 nonzero CV R2 +0.4818
alpha 1.00 7 nonzero CV R2 +0.4820
alpha 5.00 5 nonzero CV R2 +0.4659 sex,bmi,bp,s3,s5
alpha 20.00 3 nonzero CV R2 +0.3492 bmi,bp,s5
ridge alpha 1.00 10 nonzero CV R2 +0.4822
```

Embedded: L1 chooses by shrinking to zero

Lasso on load_diabetes, standardised, over a path of α :

```
alpha 0.01 10 nonzero CV R2 +0.4823
alpha 0.50 8 nonzero CV R2 +0.4818
alpha 1.00 7 nonzero CV R2 +0.4820
alpha 5.00 5 nonzero CV R2 +0.4659 sex,bmi,bp,s3,s5
alpha 20.00 3 nonzero CV R2 +0.3492 bmi,bp,s5
ridge alpha 1.00 10 nonzero CV R2 +0.4822
```

Three of ten features and half the R^2 ; five of ten and 97% of it. The lasso gives you the whole trade-off curve for free, and α **is the dial**. Ridge with the same α keeps all ten and scores the same — L1 selects, L2 shrinks.

Embedded: L1 chooses by shrinking to zero

Lasso on load_diabetes, standardised, over a path of α :

```
alpha 0.01 10 nonzero CV R2 +0.4823
alpha 0.50 8 nonzero CV R2 +0.4818
alpha 1.00 7 nonzero CV R2 +0.4820
alpha 5.00 5 nonzero CV R2 +0.4659 sex,bmi,bp,s3,s5
alpha 20.00 3 nonzero CV R2 +0.3492 bmi,bp,s5
ridge alpha 1.00 10 nonzero CV R2 +0.4822
```

Three of ten features and half the R^2 ; five of ten and 97% of it. The lasso gives you the whole trade-off curve for free, and α **is the dial**. Ridge with the same α keeps all ten and scores the same — L1 selects, L2 shrinks.

Selection here is α -dependent, so it is a hyper-parameter and belongs inside the cross-validation loop like any other.

All four on the same data — and they disagree

Ten features of thirty, load_breast_cancer:

```
pairwise overlap (Jaccard)
```

```
ANOVA F      vs mutual info    9 shared, Jaccard 0.82
```

```
ANOVA F      vs RFE            6 shared, Jaccard 0.43
```

```
ANOVA F      vs L1 logistic    4 shared, Jaccard 0.29
```

```
RFE          vs L1 logistic    6 shared, Jaccard 0.50
```

```
chosen by all four: mean concave points, worst radius, worst concave points
```

```
chosen by at least one: 16 of 30 features
```

All four on the same data — and they disagree

Ten features of thirty, load_breast_cancer:

```
pairwise overlap (Jaccard)
```

```
ANOVA F      vs mutual info    9 shared, Jaccard 0.82
```

```
ANOVA F      vs RFE            6 shared, Jaccard 0.43
```

```
ANOVA F      vs L1 logistic    4 shared, Jaccard 0.29
```

```
RFE          vs L1 logistic    6 shared, Jaccard 0.50
```

```
chosen by all four: mean concave points, worst radius, worst concave points
```

```
chosen by at least one: 16 of 30 features
```

Three features out of thirty command a consensus

The two filters agree with each other (0.82) and with nobody else. **There is no such thing as “the important features”** — there is only “the features this criterion, with this model, at this k , chose”. Report the criterion whenever you report a selection.

Does selection actually help? Measure it

selector	accuracy
all 30 features	0.9789
filter: ANOVA F, k=10	0.9596
filter: mutual info, k=10	0.9561
wrapper: RFE, k=10	0.9684
embedded: L1, C=0.08	0.9719

Does selection actually help? Measure it

selector	accuracy
all 30 features	0.9789
filter: ANOVA F, k=10	0.9596
filter: mutual info, k=10	0.9561
wrapper: RFE, k=10	0.9684
embedded: L1, C=0.08	0.9719

On this dataset, every selector lost accuracy

Not one of the four beat simply using all thirty columns. RFE, at 21 model fits per fold against a filter's 1, recovered the most. **Do not select because you were told to.** Select when you need a cheaper row, a readable model, or when d is large relative to n — which breast_cancer, at 569×30 , is not.

Does selection actually help? Measure it

selector	accuracy
all 30 features	0.9789
filter: ANOVA F, k=10	0.9596
filter: mutual info, k=10	0.9561
wrapper: RFE, k=10	0.9684
embedded: L1, C=0.08	0.9719

On this dataset, every selector lost accuracy

Not one of the four beat simply using all thirty columns. RFE, at 21 model fits per fold against a filter's 1, recovered the most. **Do not select because you were told to.** Select when you need a cheaper row, a readable model, or when d is large relative to n — which breast_cancer, at 569×30 , is not.

Reporting the experiment that did *not* work is part of doing the experiment.

Predict first #5

500 features, 60 rows, all of it drawn from $\mathcal{N}(0, 1)$. The labels are **coin flips**. There is nothing to learn: the honest answer is 0.5.

Now do what everyone does: compute the ANOVA F of every feature *on all the data*, keep the best 20, and cross-validate on those.

What accuracy comes out?

Predict first #5

500 features, 60 rows, all of it drawn from $\mathcal{N}(0, 1)$. The labels are **coin flips**. There is nothing to learn: the honest answer is 0.5.

Now do what everyone does: compute the ANOVA F of every feature *on all the data*, keep the best 20, and cross-validate on those.

What accuracy comes out?

```
select on ALL the data, then cross-validate : 0.8475 +/- 0.0293
select INSIDE each fold (Pipeline)           : 0.5342 +/- 0.0710
overstatement                                +0.3133
the wrong way beat 0.5 on 20 of 20 repetitions
```

Predict first #5

500 features, 60 rows, all of it drawn from $\mathcal{N}(0, 1)$. The labels are **coin flips**. There is nothing to learn: the honest answer is 0.5.

Now do what everyone does: compute the ANOVA F of every feature *on all the data*, keep the best 20, and cross-validate on those.

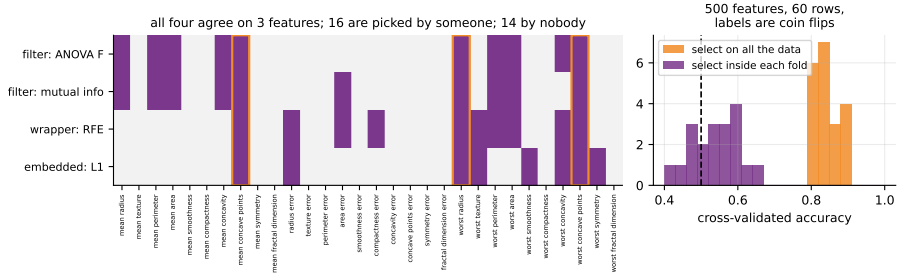
What accuracy comes out?

```
select on ALL the data, then cross-validate : 0.8475 +/- 0.0293
select INSIDE each fold (Pipeline)           : 0.5342 +/- 0.0710
overstatement                                +0.3133
the wrong way beat 0.5 on 20 of 20 repetitions
```

0.85 on data that contains no information at all

With 500 candidates and 60 rows, some feature correlates with the labels by luck — and you *chose it using the labels of the rows you are about to test on*. Every fold's “held-out” rows had already voted. +0.31, **on twenty repetitions out of twenty**.

Drawn: what each selector picks, and the leak



Left: the four selectors against the thirty features; orange boxes mark unanimity. Right: the two histograms are the *same experiment* on the *same random data*, differing only in whether the selector was fitted inside the fold. The dashed line is the truth.

The pipeline that cannot leak

```
pipe = Pipeline([
    ("scale", StandardScaler()),
    ("select", SelectKBest(f_classif, k=15)),
    ("pca", PCA(n_components=8)),
    ("model", LogisticRegression(max_iter=5000)),
])
cross_val_score(pipe, X, y, cv=5).mean()
```

```
scale -> select 15 -> PCA to 8 -> logistic regression
5-fold accuracy 0.9490 +/- 0.0140
compare: raw logistic regression, all 30 features 0.9789
```

The pipeline that cannot leak

```
pipe = Pipeline([
    ("scale", StandardScaler()),
    ("select", SelectKBest(f_classif, k=15)),
    ("pca", PCA(n_components=8)),
    ("model", LogisticRegression(max_iter=5000)),
])
cross_val_score(pipe, X, y, cv=5).mean()
```

```
scale -> select 15 -> PCA to 8 -> logistic regression
5-fold accuracy 0.9490 +/- 0.0140
compare: raw logistic regression, all 30 features 0.9789
```

The mean, the standard deviation, the fifteen chosen columns and the eight rotations are all recomputed inside every fold, from four fifths of the data. **And note the second line: the four-step pipeline is worse than doing nothing.** It cannot leak, and it also did not help.

Summary — twelve lines

1. A representation is good only relative to a model. One product column took a linear model from $R^2 = 0.9091$ to 0.9981; a tree got there unaided.
2. Binning gained the linear model +0.0957 and cost the tree -0.0323 — and made both fit exactly the same function.
3. PolynomialFeatures degree 3 turns 30 columns into 5455; on diabetes, degree 2 cost -0.0643 of R^2 .
4. Age against income: the income column supplied 99.995% of the squared distance, and standardising **changed which customer was the nearest neighbour**.
5. Scaling was worth +0.0561 to an RBF SVM, +0.0334 to k -NN, +0.0246 to logistic regression, and **exactly** 0.0000 to a tree and a forest — the same tree, node for node.
6. Unscaled, gradient descent diverged at every usable step size. Standardised, it reached least squares in 14 epochs.
7. One typo squeezed the good data 78× under StandardScaler, 71× under MinMaxScaler, and 2× under RobustScaler.
8. Ordinal-coding a nominal variable: $R^2 = 0.0743$ against 0.9868 for one-hot. The hand calculation predicted 0.0577.
9. Intercept plus all k dummies is rank-deficient: condition number 3.4×10^{15} , and two completely different coefficient vectors give identical predictions.
10. PCA on unscaled data: PC1 explained 98.20% and was the widest column. Standardised: 44.27%, and 95% needed 10 of 30 components (but 40 of 64 on digits).
11. PCA is a rotation, not a selection — 26 of 30 loadings exceed 0.10 — and it is unsupervised: it took an accuracy of 0.9850 down to 0.5200 by discarding the only informative direction.
12. **Selecting features on all the data before cross-validating scored 0.8475 on pure coin flips.**
Inside a Pipeline: 0.5342.

Before the next lecture

Read

notes.pdf — the full eigen-decomposition worked by hand, the reconstruction-error identity, and **10 practice questions with worked solutions**.

Do

Questions 3, 6 and 9. Question 6 is PCA on a 2×2 covariance matrix by hand; question 9 is a debugging story you will meet for real.

Next: splitting and validation

Train/test strategies, cross-validation, and imbalanced data. Carry one thing across: **the last slide of this lecture is the first slide of that one**.

And a habit to start now

Every preprocessing step goes in the Pipeline. If you find yourself calling `fit_transform` on `X` before splitting, stop and ask what that step just learned.

Materials: <https://github.com/tali7c/Applied-Machine-Learning>

Sources: Han, Kamber and Pei, *Data Mining: Concepts and Techniques* (3e), Ch. 3.; Müller and Guido, *Introduction to Machine Learning with Python*, Ch. 3–4.; James, Witten, Hastie, Tibshirani and Taylor, *An Introduction to Statistical Learning with Python*, Ch. 5–6, 12.; Scikit-learn documentation (preprocessing, model selection), accessed 2026-08-04.